

Roadmap: a matemática e os algoritmos por trás de kNN, clustering e filtragem colaborativa

Esses três temas formam um pacote coeso: todos representam dados como **vetores num espaço**, medem **distância/similaridade** entre eles e, nos casos mais avançados, **otimizam** uma função para aprender esse espaço. Aprenda a base uma vez e ela serve para os três.

O roadmap está em camadas. Estude de cima para baixo: cada camada usa a anterior.

Camada 0 — Pré-requisitos matemáticos

0.1 Álgebra linear (a mais importante de todas)

Tudo aqui é vetor e matriz. Se você só puder estudar uma coisa, estude isto.

- **Vetor e espaço vetorial.** Uma música/usuário vira um vetor $x = (x_1, x_2, \dots, x_n)$. "Parecido" = "perto" nesse espaço.
- **Norma (tamanho do vetor).** $\|x\| = \sqrt{x_1^2 + x_2^2 + \dots + x_n^2}$
- **Produto escalar (dot product)** — base da similaridade e da fatoração: $x \cdot y = x_1y_1 + x_2y_2 + \dots + x_ny_n$
- **Matrizes e multiplicação de matrizes.** A matriz usuário $\times$ item (R) é o objeto central da filtragem colaborativa.
- **Decomposições matriciais:**
 - **Autovalores/autovetores** (base do PCA).
 - **SVD (Singular Value Decomposition):** $A = U \Sigma V^T$. É o coração teórico da fatoração de matrizes.

0.2 Cálculo (só o necessário para otimização)

Você não precisa de cálculo pesado, mas precisa do suficiente para entender *como os modelos aprendem*.

- **Derivada e derivada parcial** ($\partial f / \partial x$).
- **Gradiente** (∇f): o vetor das derivadas parciais; aponta para onde a função cresce mais.
- **Regra da cadeia.**
- **Otimização:** o que é um mínimo de uma função e como o **gradiente descendente** chega lá: $\theta \leftarrow \theta - \eta \cdot \nabla f(\theta)$ (η = taxa de aprendizado)

0.3 Probabilidade e estatística

- Média, variância, desvio padrão.
 - Covariância e correlação de Pearson (a correlação é uma das medidas de similaridade).
 - Distribuição normal (gaussiana), inclusive a versão multivariada — necessária para o clustering por GMM.
 - Noções de Bayes (úteis para GMM e além).
-

Camada 1 — Distância e similaridade (o núcleo compartilhado)

É o conceito que liga os três temas. Aprenda cada métrica, sua fórmula e *quando* usá-la.

- **Distância euclidiana** (a "linha reta"): $d(x, y) = \sqrt{\sum_i (x_i - y_i)^2}$
- **Distância de Manhattan** (soma das diferenças absolutas): $d(x, y) = \sum_i |x_i - y_i|$
- **Distância de Minkowski** (generaliza as duas acima, com parâmetro p): $d(x, y) = (\sum_i |x_i - y_i|^p)^{1/p}$ (p=2 → euclidiana, p=1 → Manhattan)
- **Similaridade do cosseno** (mede ângulo, ignora magnitude — ótima para recomendação): $\cos(x, y) = (x \cdot y) / (\|x\| \cdot \|y\|)$
- **Correlação de Pearson** (cosseno aplicado a vetores centrados na média; corrige "usuários durões vs. generosos"): $\text{sim}(u, v) = \frac{\sum_i (r_{ui} - \bar{r}_u)(r_{vi} - \bar{r}_v)}{[\sqrt{\sum_i (r_{ui} - \bar{r}_u)^2} \cdot \sqrt{\sum_i (r_{vi} - \bar{r}_v)^2}]}$
- **Índice de Jaccard** (para dados binários: curtiu/não curtiu, comprou/não comprou): $J(A, B) = |A \cap B| / |A \cup B|$

Pré-processamento que muda tudo: normalização e padronização (z-score: $z = (x - \mu) / \sigma$). Sem isso, uma feature em escala grande domina a distância e estraga kNN e k-means. Aprenda isto junto com as métricas — é onde mais gente erra.

Camada 2 — kNN (k-Nearest Neighbors)

O algoritmo mais direto que usa só a Camada 1.

- **Ideia:** para prever algo sobre um ponto, olhe os (k) pontos mais próximos e agregue.
 - **Classificação:** voto da maioria entre os k vizinhos.
 - **Regressão / nota prevista:** média (geralmente **ponderada pela proximidade**): $\hat{y} = \frac{\sum_j w_j y_j}{\sum_j w_j}$, com peso $(w_j = 1/d(x, x_j))$ ou $(w_j = \text{sim}(x, x_j))$
 - **Escolha de k :** k pequeno = sensível a ruído; k grande = "borra" as fronteiras. Escolha por validação.
 - **Maldição da dimensionalidade:** em muitas dimensões, as distâncias ficam todas parecidas e o kNN perde força. Entenda *por que* — motiva redução de dimensionalidade.
 - **Eficiência:** força bruta é $O(n)$ por consulta. Estude **KD-trees**, **Ball-trees** e, para escala real, **ANN (Approximate Nearest Neighbors)** — é exatamente o que o "two-tower" usa para recuperar candidatos.
-

Camada 3 — Clustering (agrupamento não supervisionado)

Aqui você "alimenta" o sistema descobrindo grupos sem rótulos. Estude pelo menos estes quatro, em ordem.

3.1 k-means

- **Função objetivo** (inércia / WCSS — soma das distâncias internas ao quadrado): $J = \sum_k \sum_{\{x \in C_k\}} \|x - \mu_k\|^2$
- **Algoritmo de Lloyd** (repete até convergir):
 1. atribui cada ponto ao centróide mais próximo;
 2. recalcula o centróide: $\mu_k = (1/|C_k|) \sum_{\{x \in C_k\}} x$
- **Escolher k :** método do cotovelo (elbow) e **silhouette score**.
- **Limitações:** assume clusters esféricos e de tamanho parecido (lembre do exemplo das bandas).

3.2 Clustering hierárquico

- Constrói uma árvore (dendrograma) de subgrupos.
- Estude os tipos de **linkage** (single, complete, average, Ward) — é a regra de como medir distância entre grupos.

3.3 DBSCAN (baseado em densidade)

- Agrupa por **densidade**; parâmetros (eps) (raio) e (minPts) (mínimo de vizinhos).
- Vantagem: encontra clusters de formato arbitrário e marca **outliers** sozinho.

3.4 GMM (Gaussian Mixture Models) + algoritmo EM

- Modela os dados como **mistura de gaussianas**: $p(x) = \sum_k \pi_k \cdot N(x \mid \mu_k, \Sigma_k)$
- **Algoritmo EM (Expectation-Maximization)**: alterna entre estimar a qual gaussiana cada ponto pertence (passo E) e reestimar (π_k, μ_k, Σ_k) (passo M). É clustering "suave" (probabilístico) em vez de rígido.

Ponte com redução de dimensionalidade: estude PCA aqui (usa autovalores/SVD da Camada 0). Ajuda contra a maldição da dimensionalidade e melhora kNN e clustering.

Camada 4 — Filtragem colaborativa

Junta tudo: distância (Camada 1) na versão *memory-based*, e otimização (Camada 0) na versão *model-based*.

4.1 Memory-based (baseada em vizinhança)

É kNN aplicado à matriz (R) de notas usuário $\times$ item.

- **User-based**: acha usuários parecidos com você e prevê sua nota como média ponderada:
$$\hat{r}(u, i) = \bar{r}_u + \frac{\sum_{v \in N} \text{sim}(u, v) \cdot (r(v, i) - \bar{r}_v)}{\sum_{v \in N} |\text{sim}(u, v)|}$$
- **Item-based**: mesma ideia, mas com similaridade entre *itens* (mais estável e escalável; foi o que popularizou o "quem ouviu X ouviu Y").

4.2 Model-based: fatoração de matrizes (o pulo do gato)

- **Ideia**: aproximar a matriz gigante e esparsa (R) por duas matrizes de fatores latentes (embeddings): $R \approx P \cdot Q^T$, logo $\hat{r}(u, i) = p_u \cdot q_i$ (produto escalar do vetor do usuário pelo do item)
- **Função de custo** (erro quadrático nos dados observados + regularização para não decorar):
$$\min \sum_{(u, i) \in \text{obs}} (r(u, i) - p_u \cdot q_i)^2 + \lambda (\|p_u\|^2 + \|q_i\|^2)$$
- **Otimização — duas formas clássicas**:
 - **SGD (gradiente descendente estocástico)**. Com erro $e(u, i) = r(u, i) - p_u \cdot q_i$:
$$p_u \leftarrow p_u + \eta \cdot (e(u, i) \cdot q_i - \lambda \cdot p_u)$$

$$q_i \leftarrow q_i + \eta \cdot (e(u, i) \cdot p_u - \lambda \cdot q_i)$$
 - **ALS (Alternating Least Squares)**: fixa (Q) , resolve (P) por mínimos quadrados; fixa (P) , resolve (Q) ; repete. Paraleliza bem.
- **Feedback implícito** (cliques/plays em vez de notas): estude **Weighted ALS** e **BPR** (Bayesian Personalized Ranking) — é o caso real do Spotify/YouTube.

Camada 5 — Avaliação (sem isto você não sabe se funciona)

- **Divisão treino/teste e validação cruzada (k-fold).**
 - **Erro de predição de nota:**
 - $RMSE = \sqrt{(1/N) \sum (r - \hat{r})^2}$
 - $MAE = (1/N) \sum |r - \hat{r}|$
 - **Qualidade de ranking (o que importa em recomendação):**
 - **Precision@k** e **Recall@k**
 - **MAP** (mean average precision)
 - **NDCG** (dá mais peso a acertos no topo da lista)
 - **Problema do cold start** (usuário/item novo, sem histórico) e como conteúdo + híbrido resolvem.
-

Camada 6 — Prática (onde a matemática vira código)

- **Ferramentas base:** `numpy` (álgebra linear), `scipy` (distâncias, esparsas), `pandas`.
 - **scikit-learn:** `KNeighborsClassifier/Regressor`, `KMeans`, `DBSCAN`, `AgglomerativeClustering`, `GaussianMixture`, `PCA`, `silhouette_score`.
 - **Bibliotecas de recomendação:** `surprise` (didática, fatoração de matrizes), `implicit` (ALS para feedback implícito), `LightFM` (híbrido).
 - **Projetos sugeridos, em ordem:**
 1. Implementar distância euclidiana e cosseno na mão (só numpy).
 2. kNN do zero e comparar com o scikit-learn.
 3. k-means do zero (algoritmo de Lloyd).
 4. Recomendador item-based num dataset pequeno (ex.: MovieLens 100k).
 5. Fatoração de matrizes com SGD do zero; depois comparar com `surprise`/`implicit`.
-

Ordem de estudo sugerida (resumo das dependências)



Recursos recomendados (estáveis e gratuitos em sua maioria)

- **Álgebra linear (intuição):** série "Essence of Linear Algebra" (3Blue1Brown).
- **Álgebra linear (formal):** MIT 18.06, Gilbert Strang.
- **Cálculo/otimização (intuição):** série "Essence of Calculus" (3Blue1Brown).
- **Machine learning geral:** curso de Machine Learning do Andrew Ng; notas do Stanford CS229 para a matemática.
- **Sistemas de recomendação e clustering em escala:** livro gratuito *Mining of Massive Datasets* (Leskovec, Rajaraman, Ullman) — capítulos de clustering e de recomendação.
- **Prática:** documentação do scikit-learn (guias de "nearest neighbors", "clustering") e das libs `surprise` / `implicit` / `LightFM`.